Knowledge Distillation and Deployment of a Lightweight Command-Parsing Model on a Local Cloud

Cloud Computing Assignment

Name Sameer Singh
Roll number 230101096

Repository https://github.com/Sameer6556/cc; submitted version (tag `final`): https://github.com/Sameer6556/cc/tree/final
Model releases model-v1, model-v2

Overview, architecture and how to read this report

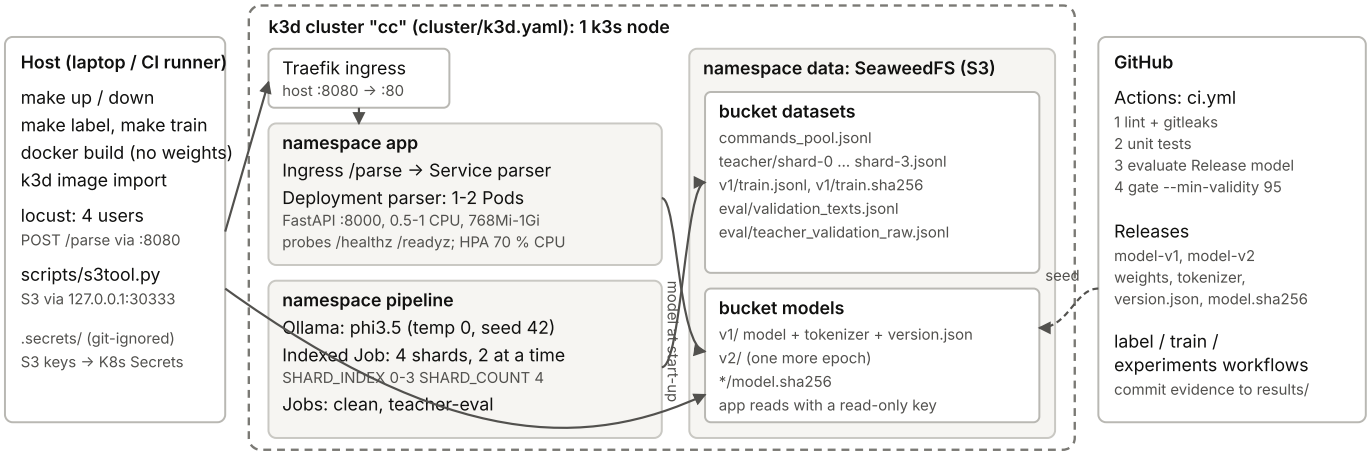


Figure 1. System architecture. Everything runs in one k3d cluster created from `cluster/k3d.yaml`; the only things outside it are the host tools and GitHub (CI and the model Releases).

The teacher (phi3.5, 3.8 B parameters, served by Ollama) labels the 3,000 pool commands with a four-key JSON frame; `clean.py` filters the replies into a training set; Pythia-70M is fine-tuned on the first 1,500 records (sequence-level distillation); the student is served by a FastAPI service behind a Traefik Ingress with an HPA. All long-running experiments were executed with the repository's own Makefile targets and scripts on GitHub-hosted runners (Ubuntu 24.04, 4 vCPU, 16 GiB RAM), from a fresh clone each time, by the workflows `label`, `train` and `experiments`, which commit their evidence to `results/`; the same commands run unchanged on a laptop (see README).

M measured: read from the file named in the caption or row A assumption or design choice, not a measurement

Task 1. Object storage and parallel teacher labelling

Architecture. The cluster is a single k3s node; Traefik is published on host port 8080 and the S3 port of SeaweedFS on 127.0.0.1:30333 only. SeaweedFS 4.47 runs as one `weed server -s3` Pod in namespace `data` on a 5 GiB volume. S3 credentials are generated at deploy time (`scripts/secrets.sh`) straight into Kubernetes Secrets: an admin identity for the pipeline and a read-only identity limited to the `models` bucket for the service (checked: GET/LIST on models allowed; GET on datasets, PUT and DELETE denied). `make up` creates the buckets `datasets` and `models` and uploads `commands_pool.jsonl`. The teacher runs inside the cluster (Ollama 0.40.9, namespace `pipeline`, phi3.5 pulled by an init container) with temperature 0 and seed 42 hard-coded in `worker/teacher.py`, plus `num_predict` 128, `num_ctx` 2048 and the stop sequences blank line and `Command:` (a frame is one line, so they only end trailing chatter early; the stored reply is still exactly what was generated).

Prompt (`prompts/teacher_prompt.txt`). It restates the four keys, gives a one-line definition per intent, the three slot rules of `schema.json` plus three conventions of the data (drop "next" but keep "this"; parts of the day and vague words such as "upcoming" are never dates or times; "near me", time zones and the names of people or of the assistant are never places), followed by 18 examples I wrote and a closing reminder. None of the 18 commands occurs in the pool or the validation set (checked by script), so no hand-made label can leak into the training data. Before the full run I compared two prompt versions on the 100 validation commands (teacher-dev workflow): the first gave 99 % valid JSON, 55 % full frame, 7.7 s per command (run), the second 100 % valid JSON, 61 % full frame, 5.9 s per command (run), so the second was used M (results/task1/prompt_iterations.json; the final in-cluster run of the same prompt and settings, on another runner, scored 62 % full frame, Task 2). A Because the prompt was chosen by looking at validation errors, the teacher's validation score is optimistic; this raises, not lowers, the bar of "half the teacher's full frame" for the student, whose score on the hidden set is unaffected.

Sharding. Worker *k* of *K* owns the commands whose 0-based line number *n* satisfies *n* mod *K* = *k*. The workers are one Kubernetes *Indexed Job* with `completions` = 4 shards of which `parallelism` = 2 run at the same time; each Pod receives its completion index through the downward API as `SHARD_INDEX`, and `SHARD_COUNT`=4. The assignment is a pure function of the line number, so the shards are disjoint and together cover every line without any coordination, locking or shared state; each worker writes only its own object `datasets/teacher/shard-k.jsonl`, so there are no write conflicts. Round-robin (rather than contiguous blocks) keeps the shards balanced even if command length drifts through the file. A worker uploads its partial object every 25 replies and, when restarted, skips the ids already present, so a Pod failure costs at most 25 teacher calls. Ollama decodes the two running workers' requests together (`OLLAMA_NUM_PARALLEL=2`). M The first attempt ran all 4 workers at once: its HTTP liveness probe (1 s timeout) failed while every core was busy generating, Kubernetes restarted Ollama 23 times and the Job gave up after 900 saved replies (pods_all_namespaces.txt in commit 2e112ab). The probes became TCP checks, concurrency was set to 2 and the run was resumed (`RESUME=1`), reusing the 900 replies. **Finding:** on this runner the teacher is compute-bound, so parallel workers did not add throughput: two concurrent workers needed 13.5 s per command each (6.8 s overall), four needed about 25 s each (6.3 s overall),

and one stream 6.2 s (teacher_eval.log). The workers bought robustness (independent shards, resumable checkpoints); more speed would need more teacher replicas on more machines. The trade-off of a static split is that a slow worker cannot hand work to a fast one.

clean.py outcome (each record counted once, first rule that applies)	records
input records (all four shards)	3,000
(a) reply is not valid JSON	5
(b) keys not exactly intent, date, time, place	2
(c) intent not one of the eight	7
(d) slot value neither null nor text of the command	187
(e) duplicate command (smallest id kept)	46
kept	**2,753**

M results/task1/clean_stats.json. input = (a)+(b)+(c)+(d)+(e)+kept is asserted in clean.py.

worker	replies	wall time	s / command	reply tokens
shard-0	750	7,119 s	13.52 s	27.3
shard-1	750	7,090 s	13.47 s	27.1
shard-2	750	7,096 s	13.47 s	27.1
shard-3	750	7,109 s	13.48 s	27.0

M resumed run, 525 new replies per shard (225 were reused); results/task1/worker-shard-*.log. Two Pods at a time: workers_running.txt.

Coverage (every id in exactly one shard, in the shard its line number dictates): **no gaps, no repeats** (coverage.txt). Re-running `clean.py` outside the cluster reproduces `train.sha256`: **match** (clean_reproduce.txt). SHA-256 of `datasets/v1/train.jsonl`: `432f46a3f35aacc0716bb2b239c87aaec6ac94eecf59949ddb4d5be3a249`
Labelling wall time: **6.9 h** over 2 attempts (2.9 h failed, 4.0 h ok).

Cleaning decisions A. A reply is parsed after trimming surrounding whitespace (as `score.py` does); a repeated key counts as (b); NaN is invalid JSON; rule (d) compares whole words, case- and space-insensitively, so "mon" does not match "monday" and an empty string never matches. Rules (a)-(d) are applied first, then duplicates among the survivors. Each kept line stores the exact `teacher_raw` reply and a `target`: the same object re-serialised with the key order intent, date, time, place (values unchanged; 2753 of 2753 kept replies were already byte-identical to it). **Limitations / improvements:** rule (d) checks form, not meaning, so a wrong-but-present span is kept; with only 4 vCPU the teacher is compute-bound, so more workers would not help; a work queue (or a Job per chunk) would remove the static-split imbalance, and a second teacher pass on rejected commands could recover data.

Task 2. Training the student (sequence-level distillation)

training/config.yaml	value
seed (fixed in code via set_seed before any randomness)	42
learning rate (AdamW, 5 % warm-up, linear decay)	0.0003
batch size	16
epochs: v1 / v2	10 / 11 (v2 = one more epoch)
maximum sequence length	128 tokens (no example truncated: 0)
libraries	Python 3.12, torch 2.14.1 (CPU), transformers 5.19.0, tokenizers 0.23.3, safetensors 0.8.0
records	first 1500 of train.jsonl (C0001 to C1627)
base model	EleutherAI/pythia-70m @ `a39f36b100fe`

M config and results/task2/train_v1.log. Training time v1 1850 s, v2 2036 s on 4 vCPU; `make train SUBSET=50` ran to the end in 62 s (train_subset50.log).

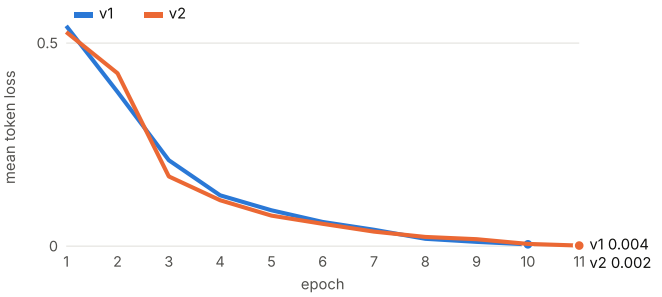


Figure 2. Mean per-token loss on the reply tokens, per epoch (training/loss.csv, loss_v2.csv) M

Implementation. Each example is `Command: <text>\nJSON: <target><|endoftext|>`. The loss is computed only on the reply and the end-of-text token (prompt tokens are masked with -100), so the student learns p(reply | command) and learns to stop. The prompt format, greedy decoding (stop at end-of-text or newline, `MAX_NEW_TOKENS=64`) and validation live in one module, `service/parser_core.py`, used by training, offline evaluation and the service, so the scores below are produced exactly the way the service answers. Training runs in deterministic mode; two runs of `make train SUBSET=50` on the same runner gave weights that are byte-identical (results/task2/determinism_check.txt). Both versions were uploaded to `s3://models/v1/` and `v2/` (results/task2/s3_listing.txt) and published as Releases whose files verify against `models/v1/model.sha256` with `sha256sum -c` (results/task2/release_v1_sha256_check.txt). Weights: `d465030ea007872b6a53abc5d2af8e709ec3dd802fe52aa030ee409426ed07ba model.safetensors`.

model (validation.jsonl, 100 commands)	JSON valid	intent	date	time	place	full frame
teacher phi3.5 (same prompt and settings as Task 1)	100.0	90.0	81.0	87.0	95.0	62.0
Pythia-70M, untrained (same prompt and decoding)	0.0	0.0	0.0	0.0	0.0	0.0
student v1 (10 epochs)	100.0	88.0	83.0	84.0	94.0	56.0
student v2 (11 epochs)	99.0	85.0	79.0	89.0	91.0	57.0

M score.py reports: results/teacher_validation.json, results/task2/untrained_validation.json, student_v1_validation.json, student_v2_validation.json (raw outputs next to them). Student v1 full frame = 90 % of the teacher's.

What changed. The untrained Pythia-70M scores 0 on every metric: after `JSON:` its first token is a line break, so under the service's decoding it returns an empty answer. After distillation, student v1 gives valid JSON for 100 % of the commands and 56 % full frame, 90 % of the teacher's 62 %. Its reply is identical to the teacher's on 66 of 100 commands. Of its 44 wrong frames, 16 repeat the teacher's mistake exactly (e.g. "upcoming" kept as a date) and 28 are its own; 13 replies contain a slot value that is not in the command at all (a mis-copied span, e.g. "dallas de's birthday" for "dale's birthday"), an error the filtered teacher data never contains. It was right on 6 commands where the teacher was wrong. One more epoch (v2) cut the final training loss from 0.0043 to

0.0016 but did not help on validation (57 % full frame, 99 % valid JSON, intent 85 % against 88 %): it starts to memorise. v1 is the deployed version (results/task2/error_analysis.txt).

Sequence-level vs logit-based distillation

(a) Sequence-level KD (Kim and Rush, 2016), used here, trains the student with ordinary cross-entropy on the teacher's *generated output sequence*: the teacher's (approximate) mode, decoded greedily, is treated as the hard target $\hat{y}$, and the student maximises $\log p_S(\hat{y} \mid x)$. **(b) Logit-based (word-level) KD** (Hinton et al., 2015) instead trains on the teacher's *full next-token probability distribution* at every position, minimising $\mathrm{KL}(p_T(\cdot \mid x, y_{<t}) \,||\, p_S(\cdot \mid x, y_{<t}))$, usually with a temperature, so the student also learns how the teacher spreads probability over the alternatives (the "dark knowledge"). **(c) It is not possible here** for two independent reasons: Ollama's API returns the generated text, not the per-position distribution over the whole vocabulary; and even with logits, phi3.5 (Llama-style SentencePiece, 32,064 tokens) and Pythia (GPT-NeoX BPE, 50,304 tokens) tokenise text differently, so their distributions are over different vocabularies and different token boundaries and cannot be compared position by position without an alignment method. **(d) Limitation of what I did:** the student sees one hard answer per command, so it copies the teacher's mistakes with full confidence (for example its slot boundaries, which differ from the SLURP labels used for scoring), and it gets no signal about which commands the teacher was unsure of; the filter in Task 1 removes malformed replies but cannot remove fluent, wrong ones.

Task 3. The inference service in a container

Implementation. FastAPI (`service/app.py`), port 8000. `POST /parse` runs greedy generation, validates the output against `schema.json` with `jsonschema` (exactly the four keys, one of the eight intents, string-or-null slots, NaN rejected) and returns either HTTP 200 with only the object or HTTP 500 `{"error": "invalid_model_output"}`; both carry `X-Model-Version` and `X-Served-By` (the Pod name from the downward API). `/healthz` answers 200 while the process lives; `/readyz` is 503 until a background thread has downloaded every object under `MODEL_URI` from `S3_ENDPOINT`, verified each file against `model.sha256`, loaded the model and run one warm-up generation. Configuration comes from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT` and `MAX_NEW_TOKENS`; credentials only from the Secret; `FAULT_INJECT=1` replaces the output with `not json` before validation. One generation runs at a time per Pod (a lock), and torch uses as many threads as the container's CPU limit (read from the cgroup). The image (`service/Dockerfile`) installs the CPU-only torch wheel, then every other package from a fully pinned lock with `--no-deps` and `pip check`; it runs as a non-root user and contains no weights.

check	result	evidence (results/task3/)
image size (limit 2 GB)	1.27 GB	image.txt
model files inside the image	none	no_weights_in_image.txt
/readyz 503 while loading, then 200; /healthz 200 throughout	11 samples at 503 before 200	readiness.log, endpoints.txt
50 sequential requests at `--cpus 1 --memory 1g`	50/50 completed, codes {200: 50}; median 365 ms, max 440 ms	sequential_50.txt
peak memory, OOM-killed	**658 MiB** of 1,024 MiB (cgroup `memory.peak`, includes page cache); highest `docker stats` sample 381 MiB; OOMKilled=false	memory_peak.txt, docker_stats.log
`FAULT_INJECT=1`	HTTP 500 with the exact error body	fault_inject.txt

M all rows.

Limitations / improvements. torch dominates the image size and start-up time; exporting the model to ONNX Runtime would shrink both. One request at a time per Pod caps throughput at one CPU's worth of decoding; batching concurrent requests would raise it. Invalid outputs are refused, not repaired; constrained decoding against the JSON schema would make every answer valid by construction.

Task 4. Kubernetes deployment, scaling and reliability

Implementation (`k8s/app/parser.yaml`). Namespace `app`: a Deployment starting at 2 replicas (requests 500m CPU / 768 Mi, limits 1 CPU / 1 Gi; readiness probe `/readyz` every 2 s, liveness `/healthz` every 10 s with 6 allowed failures; rolling update `maxSurge 1`, `maxUnavailable 0`; `preStop` sleep 5 s so Traefik drops a terminating Pod before it stops; uvicorn finishes in-flight requests for up to 20 s inside a 30 s termination grace period), a ClusterIP Service, a Traefik Ingress for `/parse` only, and an `autoscaling/v2` HPA (1 to 2 replicas, 70 % average CPU of the request, scale-down window shortened to 60 s so it can be observed). The scripts in `scripts/experiments/` produce every row below.

experiment	measured result	evidence (results/task4/)
/parse through the Ingress	HTTP 200 with both headers	basics/ingress_curl.txt, resources.txt
no ready endpoints while the model loads	10, 12, 10 timeline samples in the 3 runs (1.0 s apart on average) with 0 ready endpoints after deleting all Pods; the Service sends no traffic to a loading Pod	coldstart/run*_endpoints.log
HPA under load (4 users, 4 min)	replicas 1 → 2 → 1; second replica requested 31 s and Ready 37 s after the load started, back to one 73 s after it stopped (60 s window); 0 failed of 1130	hpa/hpa.log, timeline.log, describe_hpa.txt
rolling update v1→v2, then `rollout undo`, under load	**0.17 %** failed (2 of 1202): both are HTTP 500, the service's answer to an invalid model output, at 19:10:10 and 19:10:32 UTC, when only the two v2 Pods were serving (rollout complete 19:09:56, undo 19:10:41); v2 gives invalid JSON for 1 of the 100 validation commands that the load test cycles through. No request failed because of the Pod replacement itself; answers per version v1 915, v2 287; after rollback 5 of 5 answers say v1	rolling/rolling.log, loadtest_failures.csv, timeline.log, version_timeline.log, after_rollback.txt
delete one Pod under load	back to 2 Ready after 10.8 s; **0.00 %** failed (0 of 840)	recovery/recovery.log, timeline.log
cold start: delete all Pods, 3 runs	10 s, 12 s, 11 s; **median 11 s** (Pod creation to Ready); each Pod downloads 285 MB in 1.6 to 3.6 s from `s3://models/v1/` and loads it, with one warm-up generation, in 5.6 to 5.8 s	coldstart/summary.txt, run*_*.log
load balancing, exactly 2 Ready replicas	successful requests per replica: 50.2 %, 49.8 % (each must be ≥ 30 %)	loadbalance/loadbalance.log

M all rows. For the recovery and load-balancing runs the HPA minimum was held at 2 so that two replicas existed; it was set back to 1 afterwards.

Limitations. A The cluster is one k3s node in one container on one machine: the experiments show recovery from a lost *Pod*, not from a lost node, disk or machine; there is no fault tolerance across machines, and SeaweedFS is a single replica on a single volume. The HPA reacts to metrics-server samples (15 s) plus a stabilisation window, so it lags short bursts; and the replica count is capped at 2 by the assignment. **Improvements:** 3+ nodes (k3d agents) with pod anti-affinity, a PodDisruptionBudget, SeaweedFS replication, a startup probe instead of a long liveness delay, and an HPA on request latency rather than CPU.

Task 5. Subnetting and network access control

subnet	network / prefix	usable range	broadcast	hosts
app (1,000)	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
public (250)	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data (100)	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management (20)	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

Working in network/subnets.md: largest first, smallest prefix with $2^{32-p}-2 \geq$ need (one bit smaller gives 510, 126, 62, 14: too few); each block starts right after the previous one. Verified with Python ipaddress in tests/test_network.py.

dir	rule	match	action
in	50	all, 198.51.100.0/24	deny
in	100 / 110	tcp 80 / tcp 443, 0.0.0.0/0	allow
in	120	tcp 22, 10.20.5.128/27 (management)	allow
in	130	tcp 1024-65535, 10.20.0.0/22 (app replies)	allow
out	100	tcp 8000, 10.20.0.0/22 (app)	allow
out	110	tcp 1024-65535, 0.0.0.0/0 (client replies)	allow
both	32766	all, 0.0.0.0/0	deny

M nacl_sim.py on packets_sample.jsonl: 6 / 6 correct; 16 / 16 on my edge cases (results/network/nacl_check.txt).

Why stateless rules need reply rules, and why order matters. A NACL keeps no connection state: every packet, including a reply, is evaluated on its own. When a client opens a connection to the load balancer on 443, the answer leaves the subnet towards the client's ephemeral port (1024-65535); without outbound rule 110 every response would be dropped. Likewise the load balancer's request to the app goes out on TCP 8000, but the app's answer arrives on one of the load balancer's ephemeral ports and needs inbound rule 130 (a stateful security group would allow both automatically). Rules are checked from the lowest number and the first match wins. A packet from 198.51.100.9 to port 443 matches both the deny and the "443 from anywhere" allow, so the deny only works if its number is lower than the allow it must override: renumbering it from 50 to 140 lets that packet in (results/network/nacl_wrong_order_check.txt, 5 / 6). Limitation: a stateless NACL cannot tell a reply from a new connection, so outbound rule 110 also permits new connections to any high port.

Task 6. Reproducibility, testing and CI/CD

Reproducibility. From a fresh clone the whole system comes up with three commands (`git clone`, `cd cc`, `make up`): cluster, Secrets, SeaweedFS, buckets and dataset, model Releases downloaded, verified and uploaded, image built and imported, app deployed, smoke request. `make down` deletes the cluster. The `experiments` workflow runs exactly this on a clean runner: `make up` took 3.0 min (results/task6/make_up.log) and `make down` left no clusters (make_down.log). Python dependencies are pinned (service image: full lock); the cluster image, SeaweedFS and Ollama tags are pinned.

ci.yml stage (in order)	what it does
1. lint + secret scan	`ruff check .`; gitleaks 8.30.1 over the full Git history (must report nothing; `.gitleaks.toml` allowlists one documented false positive, the public CPython signing-key fingerprint in the `docker history` evidence)
2. unit tests	54 pytest tests: clean.py rules, sharding, service API and headers, schema validation, subnets, NACL
3. model evaluation	downloads the Release `model-v1`, checks `sha256sum -c model.sha256`, predicts validation.jsonl, runs score.py
4. quality gate	`score.py --min-validity 95`; fails the pipeline below 95 % JSON validity

M passing run: https://github.com/Sameer6556/cc/actions/runs/41505313666; run with the untrained Pythia-70M failing at the gate: https://github.com/Sameer6556/cc/actions/runs/41422191466.

The AI tools used, and how their output was checked, are described in `AI_USE.md`. **Limitation:** CI evaluates on the 100 validation commands only and cannot reproduce the multi-hour labelling; **improvement:** a nightly job that re-runs `make up` and the load test, and a held-out split of the pool.

Task 7. Performance, cost and review

standard load test (4 users, 2 min, via Ingress)	value
requests / failed	588 / 0
error rate	**0.00 %**
p50 latency	**770 ms**
p95 latency	**1300 ms**
throughput	**4.94 req/s**
mean replicas during the run	2
cold start, median of 3 (Task 4)	11 s

M results/loadtest_stats.csv, results/task7/summary.json, timeline.log.

Pod (kubectl top, every 10 s)	CPU mean / max	memory mean / max	n
`parser-7845cb4bf7-q5snf`	989 / 999 m	401 / 401 Mi	13
`parser-7845cb4bf7-whbt2`	999 / 1000 m	406 / 407 Mi	13

M results/task7/top_pods.log, one sample every 10 s during the run; the CPU limit is 1,000 m and the memory limit 1,024 Mi.

> **Cost per 1,000 requests** (data/rate_card.json: INR 3.50 per vCPU-hour, INR 0.45 per GiB-hour; each replica billed for its limits, 1 vCPU and 1 GiB):
> 1. per replica: 1 vCPU x 3.50 + 1 GiB x 0.45 = 3.9500 INR/hour
> 2. at the measured mean replica count: 2.00 replicas x 3.9500 = 7.9000 INR/hour; 4.942 req/s x 3600 = 17789.7 req/hour; 7.9000 / 17789.7 × 1000 = 0.4441 INR per 1,000 requests

SLO. *99% of /parse requests return HTTP 200 within 30 s* over the window *one standard load-test run (4 users, 2 minutes)*. Error budget = 1 % of 588 requests = **5.88** failed requests; used: 0 failed = **0 %** of the budget (SLO met). A The cost counts only the parser replicas at their limits (not SeaweedFS, Traefik or the control plane) and assumes the measured throughput is sustained for the whole hour.

Where the time goes. One request alone takes 365 ms (median) on 1 CPU (Task 3). In the standard test the 2 Pods each ran at 989 m or more of their 1,000 m CPU limit while memory stayed at or below 407 Mi of 1,024 Mi, so the service is CPU-bound. With 4 users and 2 Pods, each Pod usually holds two requests and serves them one after the other, so the median latency (770 ms) is about twice the single-request time, and the throughput (4.94 req/s) is close to the ceiling of 2 Pods / 368 ms = 5.4 req/s. Cost follows the same bottleneck: on this rate card the CPU is 89 % of a replica's price, so lowering the memory limit to 768 Mi would save only 2.8 %, while decoding twice as fast (ONNX Runtime, int8) would halve the cost per request.

component or concern	evidence (a measured number)	limitation found	improvement proposed
Data and storage	2753 of 3000 teacher replies kept; train.jsonl checksum reproduced	rule (d) keeps fluent but wrong spans; single SeaweedFS replica	second teacher pass on rejects; replicated object store
Model quality	student full frame 56.0 %, JSON validity 100.0 %	copies the teacher's slot-boundary errors; 70 M parameters	logit KD with a same-tokenizer teacher; constrained decoding
Latency and throughput	p95 1300 ms, 4.94 req/s	one request at a time per Pod; greedy decoding on 1 CPU	request batching; ONNX Runtime / int8
Resource utilisation	under load: CPU 989 m or more of 1,000 m, memory at most 407 Mi of 1,024 Mi	CPU is the bottleneck; the working set uses well under half of the memory limit	cheaper decoding per request (ONNX Runtime, int8); a 768 Mi memory limit
Reliability and recovery	rolling update 0.17 % failed (all invalid v2 outputs, none from the rollout); Pod lost under load: 0.00 % failed; cold start median 11 s	single node: no machine-level fault tolerance	multi-node cluster, anti-affinity, PodDisruptionBudget
Security and cost	gitleaks: 0 findings; INR 0.444 per 1,000 requests	S3 over plain HTTP inside the cluster; replicas billed at full limits when idle	TLS + NetworkPolicies; scale to zero off-peak

Review. M evidence column; the other columns are my assessment.

References

1. Y. Kim, A. M. Rush. Sequence-Level Knowledge Distillation. EMNLP 2016. https://aclanthology.org/D16-1139/
2. G. Hinton, O. Vinyals, J. Dean. Distilling the Knowledge in a Neural Network. arXiv:1503.02531, 2015.
3. Y.-C. Chen et al. Distilling Knowledge Learned in BERT for Text Generation. ACL 2020.
4. E. Bastianelli et al. SLURP: A Spoken Language Understanding Resource Package. EMNLP 2020 (CC BY 4.0).
5. S. Biderman et al. Pythia: A Suite for Analyzing Large Language Models. ICML 2023; EleutherAI/pythia-70m model card.
6. Microsoft. Phi-3 Technical Report (Phi-3.5-mini-instruct); Ollama library phi3.5.
7. Hugging Face Transformers and TRL documentation (causal LM fine-tuning; GKD/distillation trainer).
8. k3d and K3s documentation; SeaweedFS (S3 API); Kubernetes docs: Deployments, Services, Ingress, probes, HPA, Secrets, Indexed Jobs.
9. Locust; gitleaks; AWS VPC User Guide, Network ACLs; Google SRE Book, Service Level Objectives.

Appendix A. Measured results vs assumptions

Measured (each number in the body is read at build time by `report/build_report.py` from these files): results/task1 (labelling, cleaning, coverage), datasets/v1/train.sha256, training/loss*.csv, results/teacher_validation.json, results/task2 (training logs, evaluations, S3 listing, Release checks), results/task3, results/task4, results/network, results/task6, results/loadtest_* and results/task7.

Assumptions and design choices (not measurements): the rate card prices are hypothetical; cost counts only the parser replicas at their limits and assumes the measured throughput holds for an hour; the SLO target (99 %) and window (one standard run) are my choices; the HPA scale-down window was shortened to 60 s for observability; the HPA minimum was raised to 2 during the recovery and load-balancing runs; validation scores are optimistic for the teacher because the prompt's conventions were chosen after looking at the kind of labels in validation.jsonl (no validation command or label is in the prompt); a GitHub-hosted runner (4 vCPU, 16 GiB) stands in for a student laptop.

Appendix B. Reproducing every number

`make up`; `make label` (Task 1); `make train`, `make train MODEL_VERSION=v2`, `make eval`, `make eval-untrained` (Task 2); `bash scripts/experiments/run_all.sh` (Tasks 3, 4, 7); `python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check` (Task 5); `make lint test` (Task 6); `python report/build_report.py && node report/render.cjs` (this report).